

Manual 44 — NIST SSDF / SP 800-218 Controlled Implementation

****Controlled baseline:**** NIST SP 800-218 SSDF v1.1 final and SP 800-218A final AI community profile. SP 800-218 Rev.1 / SSDF v1.2 remains change-watch unless reverified as final at release.

Chapter 01 — Source and version hierarchy

Maintain final-versus-draft status, authoritative URLs, publication dates, change watch, owners, and release-time reverification evidence.

Chapter 02 — SSDF scope

Define products, services, code, infrastructure, pipelines, suppliers, AI components, and lifecycle stages subject to secure-development governance.

Chapter 03 — Prepare the Organization

Establish policies, roles, resources, training, tooling, environments, risk methods, and governance needed to execute secure development.

Chapter 04 — Protect the Software

Protect source, repositories, build systems, credentials, artifacts, configurations, environments, and integrity-sensitive development assets.

Chapter 05 — Produce Well-Secured Software

Apply secure design, implementation, review, analysis, testing, dependency, configuration, and release-integrity practices throughout development.

Chapter 06 — Respond to Vulnerabilities

Operate intake, triage, analysis, remediation, disclosure, coordination, verification, metrics, and lessons-learned processes.

Chapter 07 — Governance and roles

Assign accountable owners across product, engineering, security, operations, procurement, legal, privacy, risk, and executive oversight.

Chapter 08 — Secure design

Define security requirements, misuse resistance, trust boundaries, secure defaults, data protections, privilege boundaries, and design evidence.

Chapter 09 — Threat modeling

Identify assets, adversaries, attack paths, abuse cases, dependencies, mitigations, residual risks, and review triggers.

Chapter 10 — Architecture review

Perform risk-based architecture review for trust, identity, data flow, isolation, cryptography, resilience, interfaces, and deployment assumptions.

Chapter 11 — Source control

Protect branches, commits, reviews, identities, permissions, repository settings, provenance, history, and emergency changes.

Chapter 12 — Build integrity

Secure build services, runners, compilers, dependencies, configuration, isolation, reproducibility, logging, artifacts, and privileged access.

Chapter 13 — Dependency governance

Inventory direct and transitive dependencies; assess provenance, vulnerabilities, licenses, maintenance, replacement risk, and update strategy.

Chapter 14 — SBOM and provenance

Generate and protect software bills of materials, origin metadata, build attestations, component relationships, retention, and release linkage.

Chapter 15 — Secrets

Prevent hard-coded secrets; control creation, storage, delivery, scanning, rotation, revocation, emergency response, and audit evidence.

Chapter 16 — Secure coding

Adopt language/framework-specific secure coding rules, input/output controls, error handling, memory safety, authentication, authorization, and data protections.

Chapter 17 — Code review and static analysis

Use peer review and automated analysis appropriate to risk; govern findings, suppressions, false positives, exceptions, and remediation evidence.

Chapter 18 — Dynamic and security testing

Use unit, integration, dynamic, fuzz, penetration, abuse-case, API, configuration, and other risk-based security testing with traceable results.

Chapter 19 — CI/CD controls

Protect pipeline definitions, identities, approvals, environments, artifacts, deployment rights, secrets, logs, rollback, and separation of duties.

Chapter 20 — Environment separation

Separate development, test, staging, production, signing, sensitive data, administrative access, and high-trust build activities according to risk.

Chapter 21 — Release integrity, signing, and attestation

Verify approved source, tests, dependencies, build provenance, signatures, attestations, checksums, release authorization, and distribution integrity.

Chapter 22 — Vulnerability intake

Provide internal and external reporting channels, coordinated disclosure routes, deduplication, severity intake, evidence preservation, and ownership.

Chapter 23 — Remediation and prioritization

Prioritize using exploitability, exposure, business impact, dependency reach, compensating controls, active exploitation, and customer/regulatory obligations.

Chapter 24 — Supplier assurance

Assess development suppliers, hosted services, components, build dependencies, support practices, incident duties, attestations, and corrective actions.

Chapter 25 — Software acquisition

Define secure-development requirements in acquisition, evaluate supplier evidence, risks, vulnerabilities, provenance, maintenance, and exit conditions.

Chapter 26 — Incident feedback and lessons learned

Feed incidents, vulnerabilities, near misses, red-team results, customer reports, and postmortems back into requirements, design, training, and controls.

Chapter 27 — AI-development profile controls

Apply SP 800-218A where relevant to AI model/code/data pipelines, including provenance, data dependencies, model artifacts, evaluation, misuse, and deployment safeguards.

Chapter 28 — Federal and acquisition overlay boundaries

Keep SSDF guidance distinct from agency mandates, procurement clauses, contractual obligations, sector rules, and organization-specific requirements.

Chapter 29 — Evidence and metrics

Track control adoption, pipeline coverage, vulnerability aging, secure-review rates, test results, dependency risk, provenance coverage, and exceptions.

Chapter 30 — Exceptions and continual improvement

Document exceptions, risk acceptance, compensating controls, expiry, reassessment, corrective actions, maturity improvements, and management review.

Chapter 31 — Localization and provenance

Freeze EN/es-419/pt-BR candidates; bind DOCX/PDF identities to SHA-256 and retain source-version, structure, parity, accessibility, render, security, and staging evidence.

Chapter 32 — Draft watch and release-time reverification

Immediately before publication, confirm NIST version status, predecessor Manual 43 publication, exact artifact identity, clean staged-head QA, and final registry reconciliation.